# Shiv Nadar University Chennai

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## Experiment 2

### Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

## 1. Objective

The objective of this experiment is twofold. First, to understand the working principle of a Multi-Layer Perceptron (MLP) and implement one using TensorFlow/Keras for multi-class image classification, using the Fashion-MNIST dataset to train and evaluate the network, studying the process of image preprocessing (flattening, normalization and one-hot encoding), analysing the learning behaviour of the network through training and validation curves, evaluating the classifier using standard performance metrics such as Accuracy, Precision, Recall and F1-score, and subsequently performing automated hyperparameter optimization using Randomized Search in order to obtain an improved model configuration. Second, to implement the Perceptron Learning Algorithm from scratch in Python and apply it to the XOR gate, recording the weights and bias after every individual weight update, visualizing the decision boundary at every stage of training, and analysing and identifying the pattern that prevents the Single Layer Perceptron from converging on this problem.

## 2. Background Theory

A Multi-Layer Perceptron is a class of feedforward Artificial Neural Network consisting of an input layer, one or more hidden layers and an output layer. Unlike the Single Layer Perceptron studied in Experiment 1, which can only solve linearly separable problems, the Multi-Layer Perceptron introduces hidden layers together with non-linear activation functions, enabling the network to approximate complex, non-linear decision boundaries. Each neuron in a given layer is fully connected to every neuron in the preceding layer, and information flows strictly in the forward direction during inference.

Training an MLP involves two major phases. During the forward pass, the input signal is propagated through the network, layer by layer, until the output layer produces a probability distribution over the target classes. During the backward pass, the error between the predicted output and the actual label is computed using a loss function, and this error is propagated backwards through the network using the backpropagation algorithm. The gradients obtained through backpropagation are then used by an optimizer to update the weights and biases of every layer so as to progressively reduce the loss.

The XOR (Exclusive OR) function is the classical counter-example used to demonstrate the fundamental limitation of the Single Layer Perceptron. Unlike OR, AND and NOT, the XOR function is **not linearly separable**: there is no single straight line in the two-dimensional input space that can correctly separate the positive class $\{(0,1),(1,0)\}$ from the negative class $\{(0,0),(1,1)\}$, since these two classes lie on opposite diagonals of the unit square. Because a Single Layer Perceptron can only ever represent one straight-line decision boundary, the Perceptron Convergence Theorem – which guarantees convergence in finite time – does not

apply to XOR, and the algorithm is expected to update its weights indefinitely without ever reaching a stable, zero-error configuration.

### Mathematical Formulation

For a network with $L$ layers, the pre-activation output of layer $l$ is given by

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

and the corresponding activation is

$$a^{(l)} = f\left(z^{(l)}\right)$$

where

- $a^{(l-1)}$ denotes the activation vector produced by the previous layer, with $a^{(0)} = \mathbf{x}$ representing the flattened, normalized input image of dimension 784.
- $W^{(l)}$ denotes the weight matrix connecting layer $l-1$ to layer $l$.
- $b^{(l)}$ denotes the bias vector of layer $l$.
- $f(\cdot)$ denotes the non-linear activation function of layer $l$.
- $z^{(l)}$ denotes the pre-activation (weighted sum) of layer $l$.

The final, output layer produces class probabilities using the Softmax function

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}, \qquad i = 1, 2, \ldots, 10$$

where $z_i$ is the pre-activation value corresponding to class $i$ at the output layer, and $\hat{y}_i$ is the predicted probability of class $i$.

Since the problem involves multi-class classification with one-hot encoded target labels, the network is trained by minimizing the Categorical Cross-Entropy loss

$$L = -\sum_{i=1}^{10} y_i \log(\hat{y}_i)$$

where $y_i$ is the true (one-hot encoded) label for class $i$ and $\hat{y}_i$ is the predicted probability for class $i$.

The weights of every layer are updated using gradient descent, where the gradient of the loss with respect to each weight is obtained through backpropagation

$$W^{(l)}_{new} = W^{(l)}_{old} - \eta \frac{\partial L}{\partial W^{(l)}}$$

$$b^{(l)}_{new} = b^{(l)}_{old} - \eta \frac{\partial L}{\partial b^{(l)}}$$

where

- $\eta$ is the learning rate.
- $\frac{\partial L}{\partial W^{(l)}}$ and $\frac{\partial L}{\partial b^{(l)}}$ are the gradients of the loss with respect to the weights and biases of layer $l$, computed using the chain rule.

In this experiment, the gradient-based updates are carried out using adaptive optimizers, namely Adam and RMSprop, which maintain per-parameter learning rates based on the first and second moments of the gradients, resulting in faster and more stable convergence compared to plain Stochastic Gradient Descent.

The recommended baseline architecture used in this experiment is

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

As in the previous experiment, the perceptron computes

$$z = \mathbf{w}^T\mathbf{x} + b, \qquad \hat{y} = f(z)$$

and updates its parameters whenever $\hat{y} \neq y$ according to

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}, \qquad b_{new} = b_{old} + \eta(y - \hat{y})$$

where $\eta = 0.1$ in this experiment, and the weights and bias were both initialized to zero.

### Activation Functions

An activation function introduces non-linearity into the network, allowing an MLP to learn complex mapping functions that a purely linear model cannot represent. In this experiment, several activation functions are used across the baseline model, the hyperparameter search space and the final optimized model.

$$\text{ReLU: } f(z) = \max(0, z)$$

$$\text{Tanh: } f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\text{Sigmoid: } f(z) = \frac{1}{1 + e^{-z}}$$

$$\text{Softmax: } f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The Rectified Linear Unit (ReLU) is computationally efficient and mitigates the vanishing gradient problem, which makes it the default choice for hidden layers of the baseline MLP. The Hyperbolic Tangent (Tanh) function, which was selected by the hyperparameter search for the hidden layers of the optimized model, produces zero-centered outputs in the range $(-1, 1)$, often leading to smoother gradient flow than the Sigmoid function. The Softmax function is used exclusively at the output layer to convert the raw output scores into a valid probability distribution over the ten classes.

| **Activation Function** | **Output Range** | **Typical Application** |
|---|---|---|
| ReLU | $[0, \infty)$ | Hidden Layers of the Baseline MLP |
| Tanh | $(-1, 1)$ | Hidden Layers of the Optimized MLP |
| Sigmoid | $(0, 1)$ | Candidate Activation in Hyperparameter Search |
| Softmax | $(0, 1)$ | Output Layer – Multi-class Classification |

Although ReLU, Tanh and Sigmoid were all included as candidate activation functions in the hyperparameter search space, the automated search identified Tanh as the activation function that produced the best cross-validated performance for the hidden layers, and it was consequently used in the optimized model.

### Activation Function (Experiment 4 – XOR Gate)

The Step Activation Function was used, consistent with the classical Rosenblatt Perceptron formulation.

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

## 3. Dataset Description

The experiment uses the **Fashion-MNIST Dataset**, a widely used benchmark dataset in the deep learning community, consisting of grayscale images of clothing articles. It is intended as a drop-in, more challenging replacement for the original MNIST handwritten digit dataset.

| **Property** | **Value** |
|---|---|
| Dataset Name | Fashion-MNIST Dataset |
| Source | Keras Datasets (`tensorflow.keras.datasets`) |
| Training Images | 60,000 |
| Testing Images | 10,000 |
| Image Size | $28 \times 28$ (Grayscale) |
| Flattened Feature Vector | 784 |
| Target Classes | 10 |
| Missing Values | None |
| Learning Type | Supervised Learning |
| Problem Type | Multi-class Classification |

**Target Classes**

| Class Label | Description |
|---|---|
| 0 | T-shirt/Top |
| 1 | Trouser |
| 2 | Pullover |
| 3 | Dress |
| 4 | Coat |
| 5 | Sandal |
| 6 | Shirt |
| 7 | Sneaker |
| 8 | Bag |
| 9 | Ankle Boot |

The dataset contains no missing values and each of the ten classes is represented by exactly 6,000 training images, resulting in a perfectly balanced class distribution. This balance ensures that the trained network is not biased towards any particular category during learning. Each image, originally provided as a $28 \times 28$ pixel matrix, must be transformed into a flat 784-dimensional feature vector prior to being supplied to the fully connected (Dense) layers of the MLP.

The "dataset" for this experiment is the complete truth table of the XOR gate.

| x1 | x2 | XOR Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Unlike the OR and AND gates, the positive class $(0, 1)$ and $(1, 0)$ and the negative class $(0, 0)$ and $(1, 1)$ occupy opposite diagonal corners of the input square, making this dataset non-linearly separable.

## 4. Image Preprocessing

Since Dense layers of an MLP require a one-dimensional feature vector rather than a two-dimensional image matrix, every $28 \times 28$ image is flattened into a vector of length 784

$$28 \times 28 = 784$$

For instance, a small $2 \times 2$ image matrix is flattened in row-major order as follows

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10,\ 20,\ 30,\ 40]$$

The following preprocessing steps were carried out prior to model construction.

1. Loaded the Fashion-MNIST dataset using `tensorflow.keras.datasets`.
2. Displayed ten sample images together with their class labels.
3. Flattened every $28 \times 28$ image into a 784-dimensional feature vector.
4. Normalized pixel intensities from the range $[0, 255]$ to $[0, 1]$ by dividing by 255.
5. Converted the integer class labels into one-hot encoded vectors of length 10 using `to_categorical`.

## 5. Experimental Procedure

The experiment was carried out using Python with TensorFlow/Keras in Google Colab. The implementation followed a systematic workflow beginning from dataset loading and ending with automated hyperparameter optimization and final model comparison.

### Task 1: Dataset Exploration

The Fashion-MNIST dataset was loaded directly using `tensorflow.keras.datasets.fashion_mnist`. The shapes of the training and testing partitions were verified, the number of classes and class labels were printed, ten representative sample images were displayed along with their true class names, and the class distribution across the training set was plotted.

**Output**

- Training Images Shape : (60000, 28, 28)
- Training Labels Shape : (60000,)
- Testing Images Shape : (10000, 28, 28)
- Testing Labels Shape : (10000,)
- Image Size : 28 x 28
- Number of Classes : 10
- Class Labels : [0 1 2 3 4 5 6 7 8 9]

The dataset exploration confirmed that the images were correctly loaded and that no missing or corrupted samples were present.

### Task 2: Data Preprocessing

The images were flattened, normalized and one-hot encoded as described in Section 5.

**Output**

- Flattened Training Shape : (60000, 784)
- Flattened Testing Shape : (10000, 784)
- Encoded Training Shape : (60000, 10)
- Encoded Testing Shape : (10000, 10)

The comparison of tensor shapes before and after preprocessing confirmed that every image was correctly transformed from a $28 \times 28$ matrix into a flat, normalized 784-dimensional vector, and that every integer label was correctly converted into a 10-dimensional one-hot vector.

### Task 3: Model Construction

A baseline Multi-Layer Perceptron was constructed using the Keras Sequential API with the following architecture.

- Input Layer : 784 neurons (flattened pixel vector)
- Hidden Layer 1 : Dense, 128 neurons, ReLU activation
- Hidden Layer 2 : Dense, 64 neurons, ReLU activation
- Output Layer : Dense, 10 neurons, Softmax activation

The model was compiled using the Adam optimizer, the Categorical Cross-Entropy loss function and Accuracy as the evaluation metric. The `model.summary()` output confirmed a total of 109,386 trainable parameters distributed across the three Dense layers, as reported in Section 8.

### Task 4: Model Training

The baseline model was compiled using

- Optimizer : Adam
- Loss : Categorical Cross-Entropy
- Metric : Accuracy

and trained for 20 epochs with a batch size of 32, using 20% of the training data reserved as a validation split. Training and validation accuracy and loss were recorded after every epoch in order to monitor learning progress and detect signs of overfitting.

### Task 5: Model Evaluation

After training, the baseline model was evaluated on the untouched testing set of 10,000 images. The following metrics were computed.

- Accuracy
- Precision (weighted average)
- Recall (weighted average)
- F1-Score (weighted average)
- Confusion Matrix
- Classification Report (per-class precision, recall and F1-score)

### Task 6: Hyperparameter Optimization and Model Comparison

Rather than manually selecting the network hyperparameters, an automated Randomized Search was performed using the SciKeras wrapper together with Scikit-learn's `RandomizedSearchCV`, employing 5-fold cross-validation over a wide search space covering the number of hidden layers, the number of neurons per layer, the learning rate, the batch size, the number of epochs, the optimizer, the activation function and the dropout rate. The best configuration identified by the search was then used to construct and train an optimized MLP, which was evaluated on the testing set and compared against the baseline model.

The experiment was carried out using Python in Google Colab.

### Task 7: Truth Table Definition

The complete truth table of the XOR gate was defined as a NumPy array, to be used as the training data for the perceptron.

### Task 8: Perceptron Training with Fixed Epoch Budget

Since XOR is not linearly separable, the perceptron cannot naturally converge; a fixed maximum of 10 training epochs was therefore used, with the same online, sample-by-sample update rule and learning rate ($\eta = 0.1$) as in the previous experiment. Weights and bias were initialized to zero.

### Task 9: Weight Update Logging

Every time a sample was misclassified, the weights and bias were updated immediately and the updated values were printed to the console, exactly as in the previous experiment.

### Task 10: Decision Boundary Visualization

Immediately after every weight update, the corresponding decision boundary line $w_1x_1 + w_2x_2 + b = 0$ was recorded and plotted. All recorded boundaries across the full 10 training epochs were combined into a single figure using Matplotlib subplots.

### Task 11: Non-Convergence Pattern Analysis

After training, the complete sequence of recorded $(\mathbf{w}, b)$ states was scanned programmatically to check whether any state recurred later in training. A recurring state would confirm that the perceptron has entered a repeating cycle rather than converging.

## 6. Source Code

The complete implementation was carried out in Python using Google Colab and TensorFlow/Keras, following the task-wise structure described in Section 5 (dataset loading, preprocessing, baseline model construction and training, baseline evaluation, automated hyperparameter optimization using RandomizedSearchCV, optimized model construction and training, and final evaluation).Owing to its length, the complete, executable source code has not been reproduced in full within this report. The complete Google Colab notebook, containing every code cell together with its executed output, is publicly available at the following repository link.

https://github.com/K-Mithran/Deep-Learning/blob/main/Lab2/DL_2.ipynb

The major stages of implementation included

1. Loading the Fashion-MNIST dataset.
2. Performing exploratory data analysis (sample images and class distribution).
3. Flattening, normalizing and one-hot encoding the images.
4. Constructing and training the baseline MLP.
5. Evaluating the baseline model on the testing set.
6. Performing automated hyperparameter optimization using Randomized Search.
7. Constructing and training the optimized MLP using the best hyperparameters.
8. Evaluating the optimized model and comparing it against the baseline.

## 7. Hyperparameter Optimization

Instead of manually selecting the network hyperparameters, an automated search was performed using **RandomizedSearchCV** together with the SciKeras wrapper, which allows a Keras model to be used as a Scikit-learn compatible estimator. Randomized Search was preferred over an exhaustive Grid Search because the combined search space is extremely large, and Randomized Search allows a representative sample of configurations to be evaluated within a practical time budget while still providing a reliable estimate of the best-performing region of the hyperparameter space.

The following search space was defined for the optimization process.

| **Hyperparameter** | **Candidate Values** |
|---|---|
| Hidden Layers | 1, 2, 3 |
| Hidden Neurons | 32, 64, 128, 256 |
| Learning Rate | 0.1, 0.01, 0.001 |
| Batch Size | 16, 32, 64, 128 |
| Epochs | 10, 20, 30 |
| Optimizer | SGD, Adam, RMSprop |
| Activation Function | ReLU, Tanh, Sigmoid |
| Dropout Rate | 0.0, 0.2, 0.5 |

The Randomized Search was configured to sample 15 candidate hyperparameter combinations ($n_iter = 15$), each evaluated using 5-fold cross-validation, resulting in a total of 75 individual model fits. Accuracy was used as the scoring metric, and the random state was fixed at 42 to ensure reproducibility of the sampled configurations.

| **Search Configuration** | **Value** |
|---|---|
| Search Strategy | RandomizedSearchCV |
| Number of Sampled Configurations | 15 |
| Cross-validation Folds | 5 |
| Total Model Fits | 75 |
| Scoring Metric | Accuracy |
| Random State | 42 |

Upon completion of the search, the following hyperparameter combination was identified as the best-performing configuration based on the mean cross-validation accuracy.

| **Hyperparameter** | **Best Value** |
|---|---|
| Hidden Layers | 3 |
| Hidden Neurons | 128 |
| Learning Rate | 0.001 |
| Batch Size | 32 |
| Optimizer | RMSprop |
| Activation Function | Tanh |
| Epochs | 30 |
| Dropout Rate | 0.2 |
| Best Cross-validation Accuracy | 88.74% |
| Optimization Time | 5480.85 seconds (≈91.35 minutes) |

These best-identified hyperparameters were subsequently used to construct the optimized MLP, consisting of three hidden Dense layers of 128 neurons each with Tanh activation, a Dropout layer with a rate of 0.2 following every hidden layer, and an output Dense layer of 10 neurons with Softmax activation, compiled using the RMSprop optimizer with a learning rate of 0.001.

# 8. Experimental Results

## 8.1 Dataset Exploration

The Fashion-MNIST dataset was successfully loaded using the Keras Datasets API. The dataset contains 60,000 training images and 10,000 testing images, each of size $28 \times 28$ pixels, spread uniformly across 10 classes.

| **Property** | **Observed Value** |
|---|---|
| Training Images | 60,000 |
| Testing Images | 10,000 |
| Image Dimensions | $28 \times 28$ |
| Number of Classes | 10 |
| Missing Values | 0 |
| Flattened Feature Length | 784 |

The dataset was found to be complete, correctly labeled and free of missing values. Every class was represented by exactly 6,000 training samples, confirming a perfectly balanced dataset that requires no additional class-balancing technique before training.

### 8.2 Data Preprocessing Results

After flattening, every image was transformed from a $28 \times 28$ matrix into a 784-dimensional feature vector. Pixel intensities, originally in the range $[0, 255]$, were normalized to the range $[0, 1]$, and the integer class labels were converted into one-hot encoded vectors of length 10.

| **Stage** | **Shape** |
|---|---|
| Flattened Training Features | (60000, 784) |
| Flattened Testing Features | (10000, 784) |
| One-hot Encoded Training Labels | (60000, 10) |
| One-hot Encoded Testing Labels | (10000, 10) |

This preprocessing step ensured that every image was represented in a format compatible with the Dense (fully connected) layers of the MLP, and that the pixel scale did not disproportionately influence the gradient updates during training.

### 8.3 Baseline Model

The baseline MLP was constructed with the architecture 784 $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(10, Softmax), compiled using the Adam optimizer with Categorical Cross-Entropy loss.

| **Layer** | **Output Shape / Parameters** |
|---|---|
| Hidden_Layer_1 (Dense, ReLU) | (None, 128) – 100,480 params |
| Hidden_Layer_2 (Dense, ReLU) | (None, 64) – 8,256 params |
| Output_Layer (Dense, Softmax) | (None, 10) – 650 params |
| Total Trainable Parameters | 109,386 |

The baseline model was trained for 20 epochs using a batch size of 32, with 20% of the training data reserved for validation. The training accuracy increased steadily from 82.12% in the first epoch to 93.36% by the twentieth epoch, while the training loss decreased from 0.5103 to 0.1746 over the same period. The validation accuracy improved from 85.03% to a peak in the high 89% range, stabilizing around 88–89% for the later epochs, while the validation loss exhibited a mild upward trend after around epoch 9, indicating the onset of slight overfitting as the network continued to fit the training data more closely than the validation data.

| **Parameter** | **Value** |
|---|---|
| Final Training Accuracy (Epoch 20) | 93.36% |
| Final Training Loss (Epoch 20) | 0.1746 |
| Final Validation Accuracy (Epoch 20) | 88.99% |
| Final Validation Loss (Epoch 20) | 0.3608 |
| Training Time | 97.32 seconds |

The baseline model was then evaluated on the held-out testing set of 10,000 previously unseen images.

| Metric | Value |
|---|---|
| Accuracy | 88.09% |
| Precision (weighted) | 88.20% |
| Recall (weighted) | 88.09% |
| F1-Score (weighted) | 88.02% |

The per-class classification report revealed that Trouser (99% precision, 97% recall), Sandal (99% precision, 93% recall) and Bag (97% precision, 97% recall) were classified with very high accuracy, since these categories possess visually distinctive shapes. In contrast, the Shirt class obtained the lowest performance (76% precision, 64% recall), and was frequently confused with visually similar upper-body garments such as T-shirt/Top, Pullover and Coat, which is an expected and well-documented characteristic of the Fashion-MNIST dataset.

### 8.4 Optimized Model

Using the best hyperparameters identified by the Randomized Search (Section 8), the optimized MLP was constructed with three hidden Dense layers of 128 neurons each using Tanh activation, each followed by a Dropout layer with a rate of 0.2, and compiled using the RMSprop optimizer with a learning rate of 0.001.

| Layer | Output Shape / Parameters |
|---|---|
| Hidden_Layer_1 (Dense, Tanh) | (None, 128) – 100,480 params |
| Dropout (rate = 0.2) | (None, 128) – 0 params |
| Hidden_Layer_2 (Dense, Tanh) | (None, 128) – 16,512 params |
| Dropout (rate = 0.2) | (None, 128) – 0 params |
| Hidden_Layer_3 (Dense, Tanh) | (None, 128) – 16,512 params |
| Dropout (rate = 0.2) | (None, 128) – 0 params |
| Output_Layer (Dense, Softmax) | (None, 10) – 1,290 params |
| Total Trainable Parameters | 134,794 |

The optimized model was trained for 30 epochs with a batch size of 32 and a 20% validation split. The training accuracy increased steadily from 78.76% in the first epoch to 90.33% by the thirtieth epoch, while the training loss decreased from 0.5844 to 0.2750. The validation accuracy rose from 82.98% to a final value of 88.85%, and the validation loss decreased from 0.4638 to a final value of 0.3326. Compared to the baseline model, the training and validation curves of the optimized model remained noticeably closer to each other throughout training, indicating that the additional Dropout regularization was effective in reducing the gap between training and validation performance, thereby limiting overfitting.

| Parameter | Value |
|---|---|
| Final Training Accuracy (Epoch 30) | 90.33% |
| Final Training Loss (Epoch 30) | 0.2750 |
| Final Validation Accuracy (Epoch 30) | 88.85% |
| Final Validation Loss (Epoch 30) | 0.3326 |
| Training Time | 157.47 seconds |

The optimized model was evaluated on the same testing set of 10,000 images used for the baseline model.

| Metric | Value |
|---|---|
| Accuracy | 88.05% |
| Precision (weighted) | 88.18% |
| Recall (weighted) | 88.05% |
| F1-Score (weighted) | 88.10% |

The per-class classification report of the optimized model showed a similar overall pattern to the baseline model, with Trouser (99% precision, 96% recall) and Bag (97% precision, 97% recall) remaining the best-classified categories. Notably, the Shirt class, which was the weakest class for the baseline model, improved slightly in recall (from 64% to 71%) with the optimized model, although its precision decreased marginally (from 76% to 69%), suggesting a shift in the decision boundary rather than a uniform improvement across all metrics for that class.

### 8.5 Baseline vs Optimized: Performance Comparison

| Metric | Baseline | Optimized |
|---|---|---|
| Accuracy | 88.09% | 88.05% |
| Precision | 88.20% | 88.18% |
| Recall | 88.09% | 88.05% |
| F1-Score | 88.02% | 88.10% |
| Training Time | 97.32 s | 157.47 s |

The comparison shows that the optimized model achieved a testing accuracy and precision that were marginally lower than the baseline model, while its F1-Score was marginally higher. In practical terms, the two models performed almost identically on the held-out testing set, with differences well under half a percentage point across all four metrics. This result is discussed further in Section 11.

### 8.6 Weight Update Log (Epochs 1–3)

The perceptron was trained on the XOR gate with a learning rate of 0.1 and zero initial weights and bias. The complete weight update log for the first three epochs is shown below.

| Update | Epoch | Input | Target | Pred | Weights | Bias |
|---|---|---|---|---|---|---|
| 1 | 1 | (0,0) | 0 | 1 | [0.0, 0.0] | -0.1000 |
| 2 | 1 | (0,1) | 1 | 0 | [0.0, 0.1] | 0.0000 |
| 3 | 1 | (1,1) | 0 | 1 | [-0.1, 0.0] | -0.1000 |
| 4 | 2 | (0,1) | 1 | 0 | [-0.1, 0.1] | 0.0000 |
| 5 | 2 | (1,0) | 1 | 0 | [0.0, 0.1] | 0.1000 |
| 6 | 2 | (1,1) | 0 | 1 | [-0.1, 0.0] | 0.0000 |
| 7 | 3 | (0,0) | 0 | 1 | [-0.1, 0.0] | -0.1000 |
| 8 | 3 | (0,1) | 1 | 0 | [-0.1, 0.1] | 0.0000 |
| 9 | 3 | (1,0) | 1 | 0 | [0.0, 0.1] | 0.1000 |
| 10 | 3 | (1,1) | 0 | 1 | [-0.1, 0.0] | 0.0000 |

Epoch 1 and Epoch 2 each finished with 3 misclassifications, while Epoch 3 was the first epoch to finish with 4 misclassifications – the point at which, as shown in the next subsection, the perceptron locks into a permanent repeating cycle.

### 8.7 Repeating Cycle (Epochs 4–10)

Every epoch from Epoch 3 onward produced *exactly* 4 misclassifications, updating the weights and bias through the identical sequence of states shown in Update 7 through Update 10 above. This exact 4-update sequence was found, unchanged, in every subsequent epoch through Epoch 10 (Updates 11–38), for a total of $3 + 4 \times 9 = 39$ recorded update events across 10 epochs. Training was halted after the maximum of 10 epochs without reaching a zero-error epoch.

| Property | Value |
|---|---|
| Total Weight Updates (10 epochs) | 38 |
| Misclassifications per Epoch (Epochs 3–10) | 4 (constant) |
| Zero-Error Epoch Reached | No |
| Final Recorded Weights (Update 38) | [-0.1, 0.1] |
| Final Recorded Bias (Update 38) | 0.0000 |

### 8.8 Non-Convergence Pattern Analysis

The automated cycle-detection routine applied to the recorded weight/bias history reported the following.

| **Cycle Detection Output** |
|---|
| Weight/Bias state $(-0.1,\ 0.0,\ -0.1)$ first seen at Update 3 reappears at Update 7. |
| This confirms the weights and bias enter a repeating cycle instead of converging. |

This result identifies the precise pattern that prevents convergence: starting from Update 3, the perceptron's parameters $(\mathbf{w}, b)$ cycle through the same 4 states, indefinitely, once every epoch. The root cause is that the XOR positive class $\{(0,1),(1,0)\}$ and negative class $\{(0,0),(1,1)\}$ lie on opposite diagonals of the unit square, so no single straight line can place all positive points on one side and all negative points on the other. Every time the perceptron adjusts its boundary to correctly classify one diagonal pair, it simultaneously misclassifies the other diagonal pair, so the weight updates perpetually correct one error at the cost of introducing another, producing the observed cycle rather than convergence.

## 9. Performance Summary

| | |
|---|---|
| Training Images | 60,000 |
| Testing Images | 10,000 |
| Flattened Feature Length | 784 |
| Number of Classes | 10 |
| Baseline Architecture | 784-128-64-10 |
| Baseline Optimizer | Adam |
| Baseline Epochs | 20 |
| Baseline Testing Accuracy | 88.09% |
| Hyperparameter Search Method | RandomizedSearchCV (5-fold CV) |
| Best CV Accuracy | 88.74% |
| Optimized Architecture | 784-128-128-128-10 (Dropout 0.2) |
| Optimized Optimizer | RMSprop (lr = 0.001) |
| Optimized Epochs | 30 |
| Optimized Testing Accuracy | 88.05% |

| | |
|---|---|
| Learning Rate | 0.1 |
| Weight/Bias Initialization | Zero |
| Maximum Epochs | 10 |
| Total Weight Updates | 38 |
| Converged | No |
| Misclassifications per Epoch (Steady State) | 4 of 4 samples |
| Cycle Length | 4 updates (1 epoch) |
| Cycle First Detected | State at Update 3 recurs at Update 7 |

# 10. Generated Plots with Interpretation

## 10.1 Sample Images

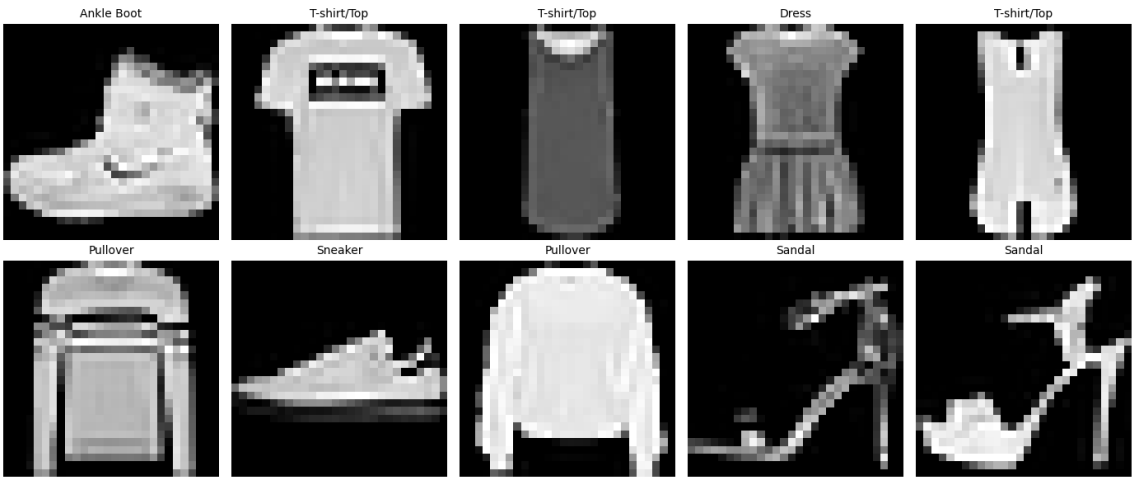



Figure 1: Ten Sample Images from the Fashion-MNIST Dataset with True Class Labels

The sample images illustrate the visual diversity of the ten clothing categories present in the Fashion-MNIST dataset. Several categories, such as Trouser and Sandal, possess distinctive silhouettes, whereas categories such as Shirt, Pullover and Coat share considerable visual overlap, which explains the relatively lower classification performance observed for these classes.

## 10.2 Class Distribution

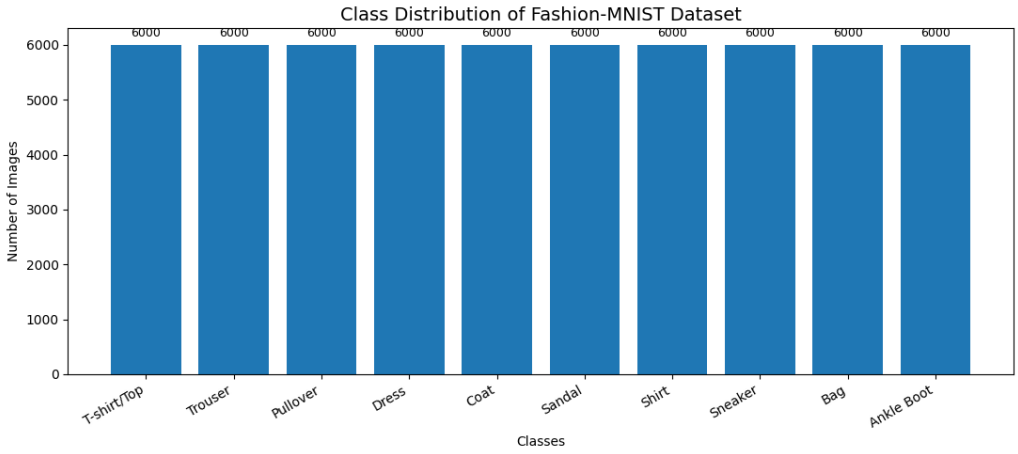



Figure 2: Class Distribution of the Fashion-MNIST Training Dataset

The bar chart confirms that each of the ten classes contains exactly 6,000 training images, indicating a perfectly balanced dataset. This balance ensures that the network is not biased towards predicting any particular class more frequently due to unequal representation in the training data.

## 10.3 Baseline Confusion Matrix

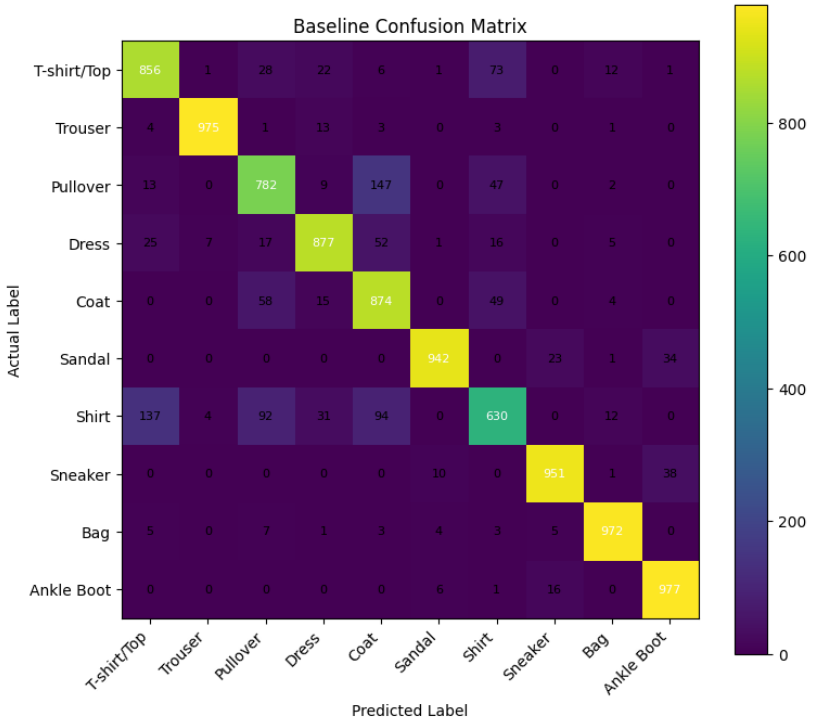



Figure 3: Confusion Matrix of the Baseline MLP on the Testing Set

The confusion matrix of the baseline model shows that the majority of predictions lie along the diagonal, confirming a high overall classification accuracy. The most prominent off-diagonal

confusion occurs between the Shirt class and the T-shirt/Top, Pullover and Coat classes, which is consistent with the visual similarity among these upper-body garment categories.

### 10.4 Baseline Training Accuracy vs Epoch

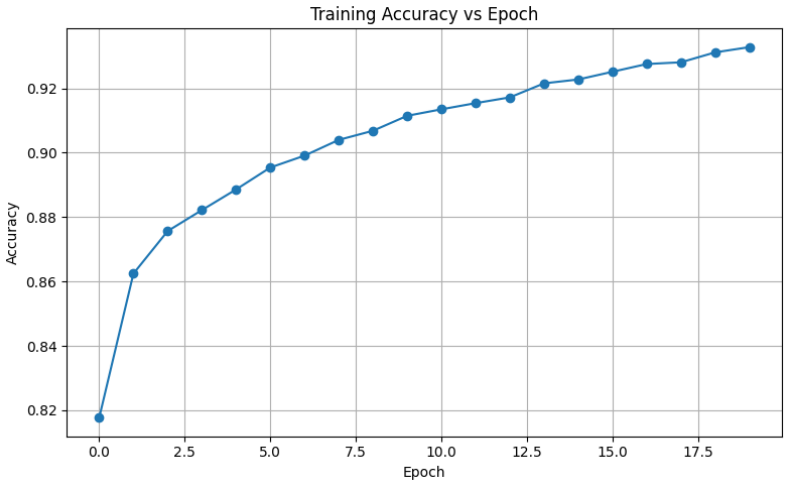



Figure 4: Training Accuracy vs Epoch – Baseline Model

The training accuracy of the baseline model increases consistently across all 20 epochs, rising from approximately 82% to over 93%. The smooth, monotonically increasing curve indicates that the network is steadily learning the mapping between the input pixel vectors and the correct class labels, without any signs of instability during training.

### 10.5 Baseline Validation Accuracy vs Epoch

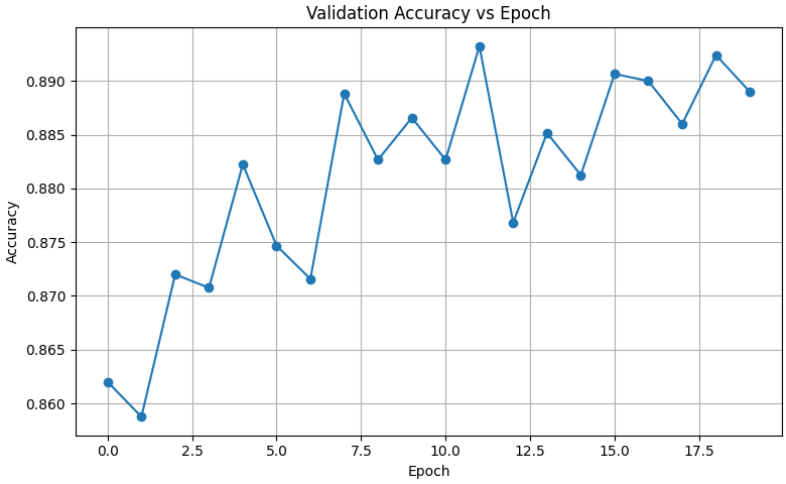



Figure 5: Validation Accuracy vs Epoch – Baseline Model

The validation accuracy rises sharply during the first few epochs and then plateaus around 88–89%, with minor epoch-to-epoch fluctuations. Unlike the training accuracy, the validation accuracy does not continue to increase towards the later epochs, suggesting that the baseline model begins to overfit the training data after roughly epoch 9 or 10.

### 10.6 Baseline Training Loss vs Epoch

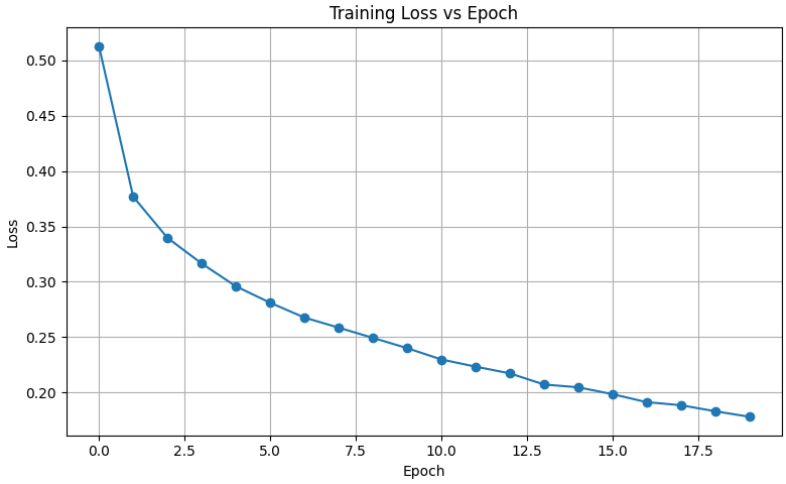



Figure 6: Training Loss vs Epoch – Baseline Model

The training loss decreases smoothly and consistently throughout all 20 epochs, falling from approximately 0.51 to below 0.18. This steady decline demonstrates that the Adam optimizer is effectively minimizing the Categorical Cross-Entropy loss on the training set at every step.

### 10.7 Baseline Validation Loss vs Epoch

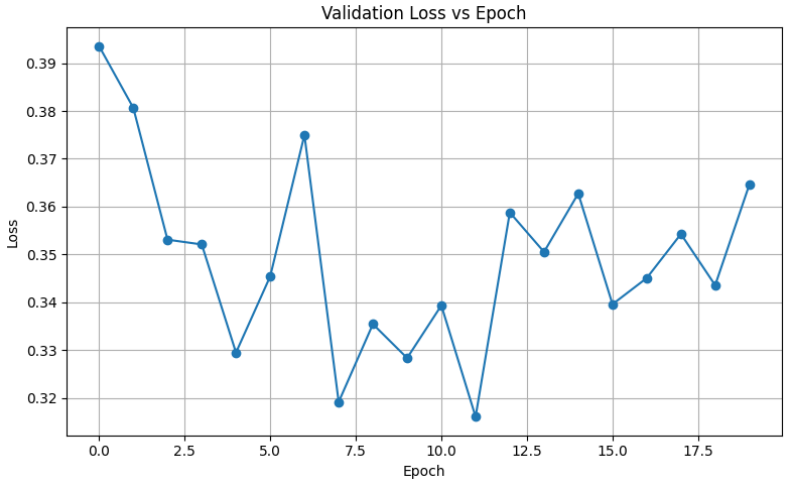



Figure 7: Validation Loss vs Epoch – Baseline Model

Unlike the training loss, the validation loss decreases only for the first few epochs before beginning to fluctuate and gradually trend upward for the remaining epochs. This divergence between the training loss (which keeps decreasing) and the validation loss (which stagnates and rises) is a classic indicator of overfitting in the baseline model, motivating the use of regularization techniques such as Dropout in the optimized model.

### 10.8 Optimized Model Accuracy

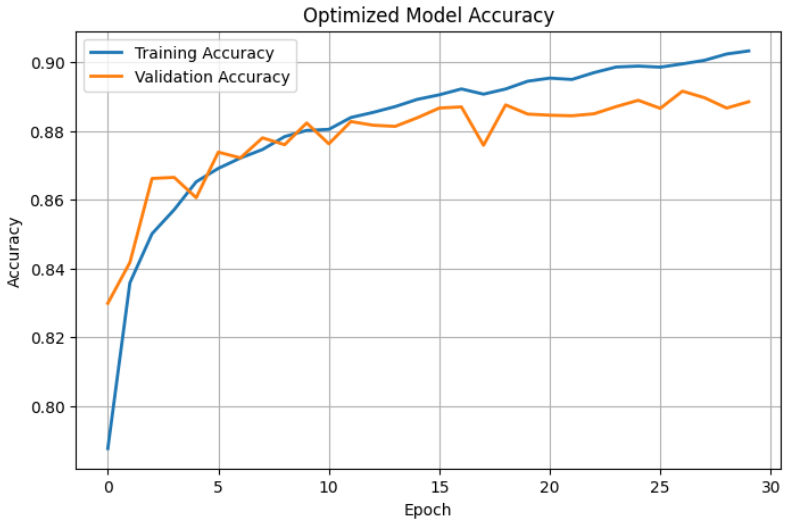



Figure 8: Training and Validation Accuracy vs Epoch – Optimized Model

The training and validation accuracy curves of the optimized model remain much closer together throughout the 30 training epochs compared to the corresponding baseline curves. Both curves rise steadily and converge towards a similar final value near 89–90%, indicating that the Dropout layers introduced in the optimized architecture successfully reduced the gap between training and validation performance.

### 10.9 Optimized Model Loss

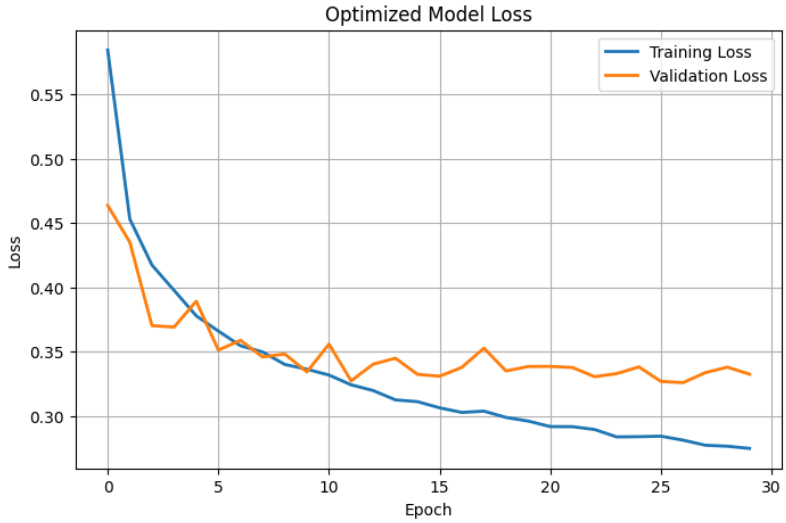



Figure 9: Training and Validation Loss vs Epoch – Optimized Model

The training and validation loss curves of the optimized model decrease together for a longer portion of the training process compared to the baseline model, and the gap between the two curves remains comparatively narrow even towards the final epochs. This behaviour confirms that the combination of additional hidden layers, Dropout regularization and the RMSprop optimizer produced a more stable and better-regularized training process.

### 10.10 Optimized Model Confusion Matrix

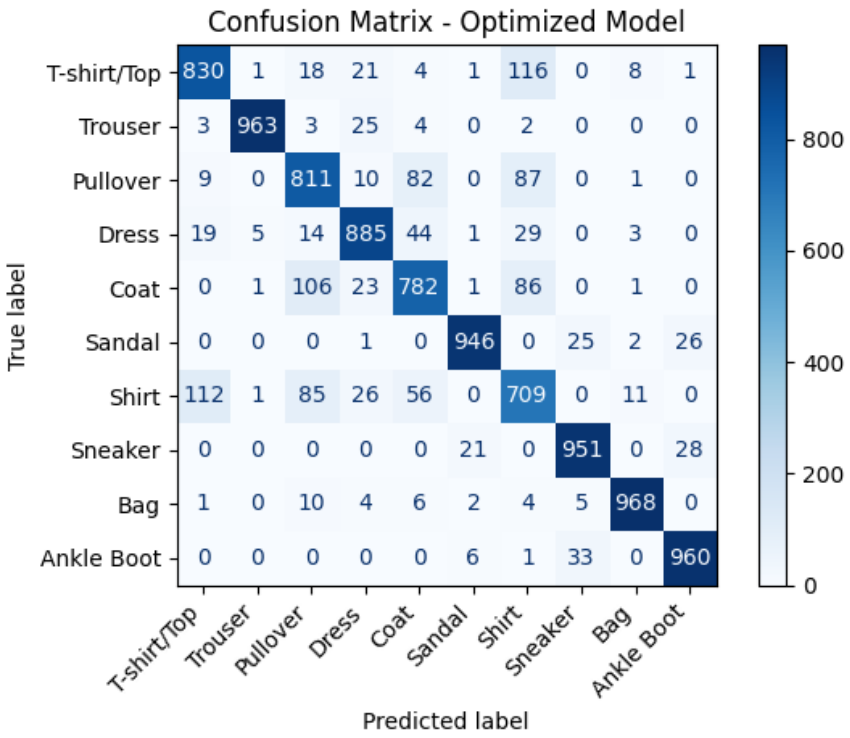



Figure 10: Confusion Matrix of the Optimized MLP on the Testing Set

The confusion matrix of the optimized model shows a diagonal pattern very similar to that of the baseline model, with the vast majority of testing samples correctly classified. The Shirt class continues to be the most frequently confused category, being misclassified primarily as T-shirt/Top, Pullover and Coat, indicating that this difficulty stems from the inherent visual ambiguity of the dataset rather than from a deficiency specific to either model architecture.

## 10.11 Combined Decision Boundary Visualization

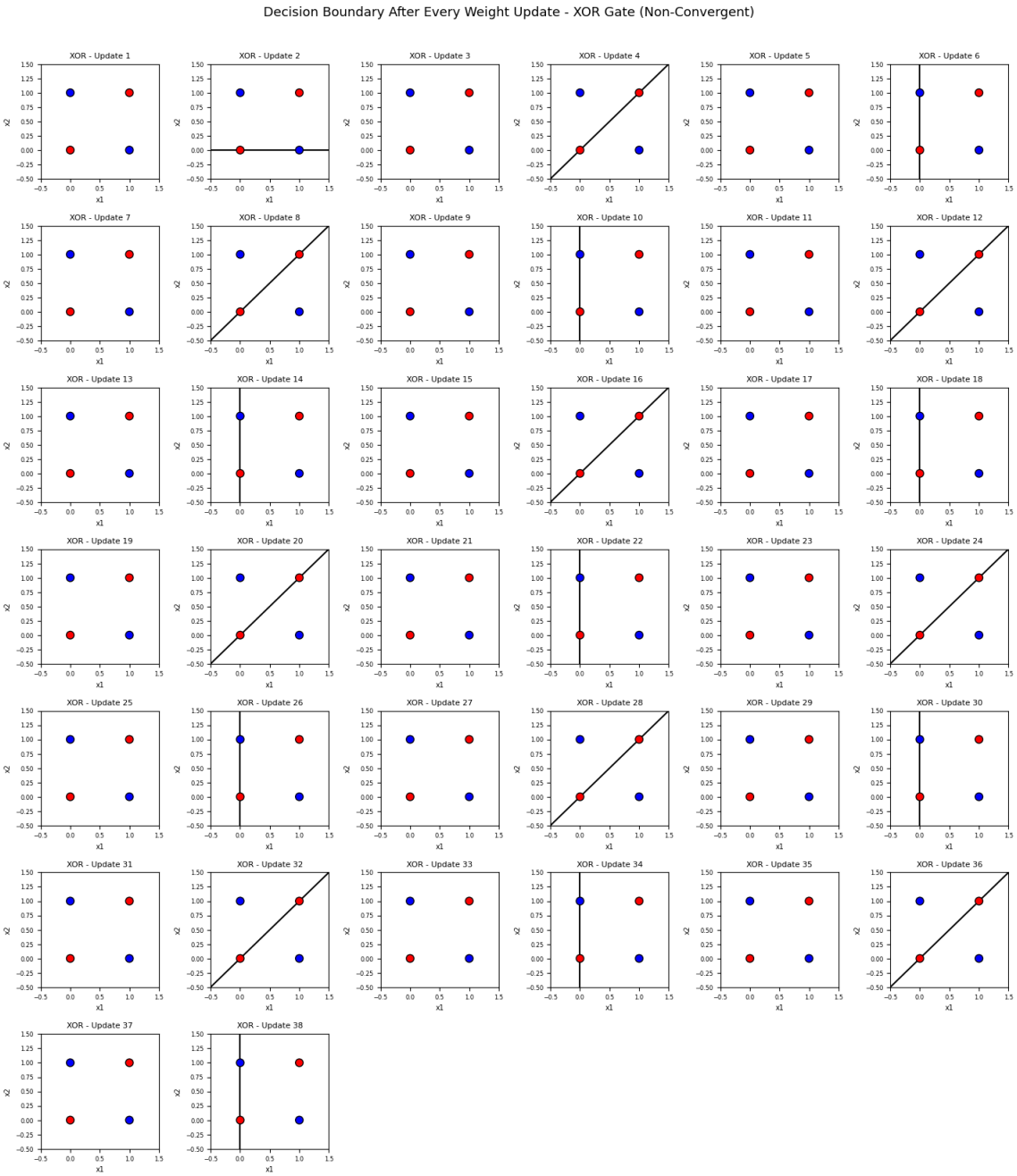



Figure 11: Decision Boundary After Every Weight Update for the XOR Gate (Non-Convergent)

The combined figure displays all 38 decision-boundary snapshots recorded across the 10 training epochs. Unlike the OR, AND and NOT gates, no consistent, stable boundary ever emerges: the plotted line is repeatedly seen alternating between only three distinct positions – a diagonal line running through the $(0, 0)$–$(1, 1)$ corners, a vertical line at $x_1 = 0$, and blank/undefined states where the boundary momentarily disappears when $w_2 \approx 0$. Rather than progressively converging towards a single fixed boundary as seen for OR and AND, the plots after Update 7 begin repeating the exact same three or four boundary positions over and over, visually confirming that the perceptron has entered a permanent oscillation instead of settling on a solution.

## 11. Discussion

The Multi-Layer Perceptron demonstrated that a fully connected feedforward network trained on flattened, normalized pixel vectors can classify Fashion-MNIST images effectively, with both the baseline and optimized models reaching a testing accuracy of approximately 88%. Randomized Search was preferred over an exhaustive Grid Search because the defined search space, spanning eight hyperparameters, would have been computationally prohibitive to evaluate exhaustively; sampling 15 configurations under 5-fold cross-validation instead located a strong region of the space at a fraction of the cost. The search selected 3 hidden layers of 128 neurons, Tanh activation, a Dropout rate of 0.2 and the RMSprop optimizer (learning rate 0.001) as the best configuration, with the number of hidden layers, the activation function and the dropout rate appearing to influence cross-validation accuracy the most.

Notably, the optimized model's testing accuracy (88.05%) was marginally lower than the baseline's (88.09%), despite the optimized configuration achieving a higher cross-validation accuracy (88.74%). This is not a failure of the search: cross-validation accuracy is estimated on resampled training folds and need not transfer exactly to a single fixed test split, the added Dropout trades a little raw fitting capacity for stability, and a difference under half a percentage point lies well within normal run-to-run variation of stochastic training. The real benefit of the optimized model is visible in its accuracy and loss curves, where the training and validation curves track much closer together than in the baseline, indicating reduced overfitting and more reliable generalization behaviour. Both models found the Shirt class hardest to classify, confusing it mainly with T-shirt/Top, Pullover and Coat, reflecting genuine visual overlap between these garments at $28 \times 28$ resolution rather than an architectural weakness, and would likely require a spatially-aware model such as a CNN to resolve.

The XOR gate experiment, carried out with a Single Layer Perceptron rather than an MLP, provides a complementary and instructive contrast to these results. The experiment confirmed that the Single Layer Perceptron cannot learn the XOR function: training never reached a zero-error epoch even after 10 full epochs (38 individual weight updates); instead, from the third epoch onward, every epoch produced exactly the same 4 misclassifications, and an automated cycle-detection check confirmed that the recorded weight/bias state at Update 3 reappeared identically at Update 7, proving that the parameters had entered a permanent repeating cycle. This behaviour is the direct consequence of XOR being non-linearly separable: its positive and negative classes occupy opposite diagonal corners of the input square, so no single straight-line boundary – the only kind of boundary a Single Layer Perceptron can represent – can ever separate them correctly, so every corrective update fixes one pair of points while simultaneously breaking the other pair, producing oscillation rather than convergence. This result is the classical illustration first highlighted by Minsky and Papert, and it directly motivates the use of Multi-Layer Perceptrons such as the one used for Fashion-MNIST classification in this report, since hidden layers and non-linear activation functions are precisely what is required to construct the piecewise, non-linear decision boundary that problems like XOR demand.

## 12. Conclusion

A Multi-Layer Perceptron was implemented in TensorFlow/Keras for multi-class classification on the Fashion-MNIST dataset, and a Single Layer Perceptron was separately implemented from scratch and applied to the XOR gate to examine the limits of linear classifiers. After flattening, normalizing and one-hot encoding the images, a baseline MLP (two hidden Dense layers) trained for 20 epochs achieved a testing accuracy of 88.09%, precision of 88.20%, recall of 88.09% and F1-Score of 88.02%. RandomizedSearchCV with 5-fold cross-validation was then used to tune the hidden layers, neurons, learning rate, batch size, epochs, optimizer, activation function and dropout rate, identifying a deeper, Dropout-regularized, Tanh-based RMSprop configuration

with a cross-validation accuracy of 88.74%; retrained with these settings, the optimized model achieved a testing accuracy of 88.05%, precision of 88.18%, recall of 88.05% and F1-Score of 88.10% – essentially on par with the baseline, but with a visibly narrower training-validation gap, reflecting improved regularization.

In parallel, the Perceptron Learning Algorithm was applied to the XOR gate, with the weights and bias logged after every individual update over 10 training epochs. Unlike linearly separable problems, the perceptron never converged on XOR: it produced 4 misclassifications in every epoch from the third epoch onward, and an automated check confirmed that its weight/bias state entered an exact repeating cycle, while the combined decision-boundary visualization showed the boundary line oscillating between the same few positions instead of settling on a stable solution.

Together, these results show that automated hyperparameter optimization does not always yield a strictly higher test accuracy than a well-designed baseline, but remains valuable for improving training stability and generalization, while the XOR experiment concretely demonstrates the fundamental limitation of the Single Layer Perceptron – its inability to solve non-linearly separable problems – and motivates the need for Multi-Layer Perceptrons with hidden layers and non-linear activation functions, of exactly the kind implemented and evaluated on Fashion-MNIST in this report. The experiment as a whole reinforced practical understanding of MLP design, activation functions, image preprocessing, backpropagation-based training and systematic model comparison using standard classification metrics.

## 13. References


1. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. Simon Haykin, *Neural Networks and Learning Machines*, Pearson Education, 2009.
4. Han Xiao, Kashif Rasul, Roland Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, 2017.

   https://github.com/zalandoresearch/fashion-mnist
5. TensorFlow/Keras Documentation.

   https://www.tensorflow.org/guide/keras
6. Scikit-learn Documentation.

   https://scikit-learn.org
7. SciKeras Documentation.

   https://www.adriangb.com/scikeras/
8. Frank Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, Psychological Review, 1958.
9. Marvin Minsky, Seymour Papert, *Perceptrons: An Introduction to Computational Geometry*, MIT Press, 1969.